



Lecture 6

Saving Data

Saving key-value sets

Saving to files

Saving to SQL databases



Saving Data

- Most Android apps need to save data, even if only to save information about the app state during `onPause()` so the user's progress is not lost
- Most non-trivial apps also need to save user settings, and some apps must manage large amounts of information in files and databases

This can be done using:

- Saving key-value pairs of simple data types in a shared preferences file
- Saving arbitrary files in Android's file system
- Using databases managed by SQLite or MySQL



1

Saving key-value sets using
`sharedPreferences`



SharedPreferences

- If you have a relatively small collection of key-values that you'd like to save, you should use the `SharedPreferences` APIs.
- A `SharedPreferences` object points to a file containing key-value pairs and provides simple methods to read and write them.
- Each `SharedPreferences` file is managed by the framework and can be private or shared.

i

Note: The **SharedPreferences** APIs are only for reading and writing key-value pairs and you should not confuse them with the **Preference** APIs, which help you build a user interface for your app settings (although they use **SharedPreferences** as their implementation to save the app settings).



sharedPreferences (example)

KEY	VALUE
highscore	200
highquality	TRUE
loggedin	1



Creating and accessing

- You can create a new shared preference file or access an existing one by calling one of two methods:

`getSharedPreferences ()`

Use this if you need multiple shared preference files identified by name, which you specify with the first parameter. You can call this from any Context in your app.

`getPreferences ()`

Use this from an Activity if you need to use only one shared preference file for the activity. Because this retrieves a default shared preference file that belongs to the activity, you don't need to supply a name.



Examples

Option 1

If we want more than 1 file, we need to access/create using some String name of our choice

```
SharedPreferences sharedPref =  
getSharedPreferences("myfile", Context.MODE_PRIVATE);
```

Option 2

If we just want 1 file, then there is no need for a name, just the mode

```
SharedPreferences sharedPref =  
getPreferences(Context.MODE_PRIVATE);
```



How to **write** to sharedPreferences?

1. To write to a shared preferences file, create a `SharedPreferences.Editor` by calling `edit()` on your `SharedPreferences`
2. Pass the keys and values you want to write with methods such as `putInt()` and `putString()`
3. Then call `commit()` to save the changes



Writing to sharedPreferences Example

```
SharedPreferences mypref=  
getPreferences (Context.MODE_PRIVATE) ;  
  
SharedPreferences.Editor e = mypref.edit() ;  
  
e.putInt ("highscore", 400) ;  
  
e.commit() ;
```



Read from SharedPreferences

- To retrieve values from a shared preferences file, call methods such as [getInt\(\)](#) and [getString\(\)](#), providing the key for the value you want, and optionally a default value to return if the key isn't present.

```
SharedPreferences sharedPref =  
getPreferences (Context.MODE_PRIVATE) ;  
  
int defaultValue = 100; //example  
  
int highScore = sharedPref.getInt ("highscore",  
defaultValue) ;
```



Example

```
public void saveoption(View view){
    SharedPreferences mypref = getPreferences(Context.MODE_PRIVATE);
    SharedPreferences.Editor e = mypref.edit();

    RadioButton r0 = (RadioButton) findViewById(R.id.radio0);
    //RadioButton r1 = (RadioButton) findViewById(R.id.radio1);

    if(r0.isChecked()){

        e.putString("choice", "option 0");
    }
    else { e.putString("choice", "option 1");}

    e.commit();

}
```



Accessing the sharedpreference file from different activities.

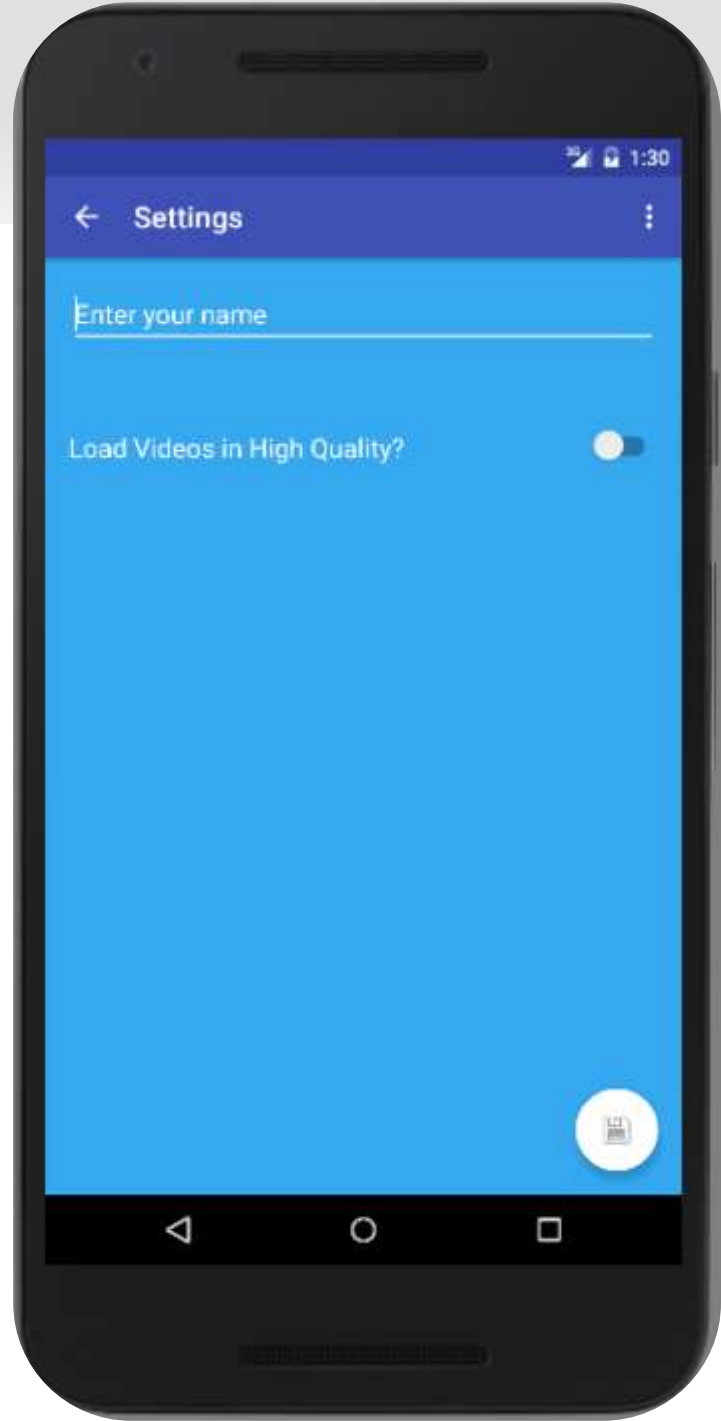
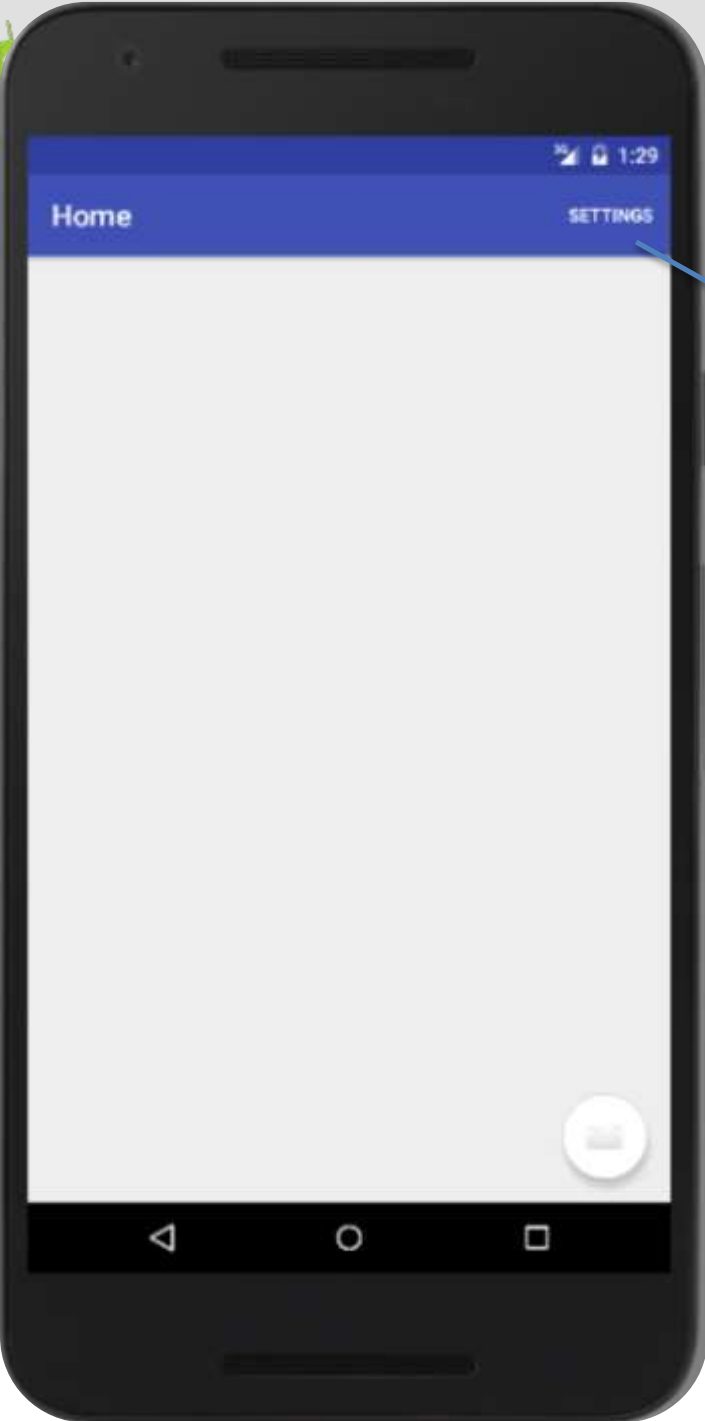
- Same as before but assign a name for it



Lab session

- Topics covered:
 - sharedPreferences
 - Floating button
 - SnackBar
 - Action Bar (from a previous chapter)

Exercise





Exercise

- 2 activities:
- MainActivity is empty but has an action bar containing “settings”
- When we press on settings, we go to the Settings Activity containing an EditText and a switch and a floating button containing an icon for saving.
- When we press on save, the values are saved.



Exercise continues

- Each time we go to the settings activity, the PREVIOUSLY saved values should be loaded.
- Of course the user can change them again and save.
- Also note that if you exit the app and enter again, the valued should be still saved.
- Extra care should be taken if we are going to the Settings activity for the FIRST time.



solution

- The solution will be 2 parts. Each having several parts.
- PART 1: Main activity.
 - Part 1.1 add the “settings” item to the action bar
 - Part 1.2 Program it to take me to the SettingsActivity when clicked. (you need to modify the `onOptionsItemSelected()`)



PART 1.1 solution

- Add the settings to the action bar in the xml file in the MENU folder. (you will find it added by default but let's make it ALWAYS instead of NEVER).

```
<menu
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
xmlns:tools="http://schemas.android.com/tools"
tools:context=".Main2Activity">

<item android:id="@+id/mySettings"
      android:title="@string/action_settings"
      android:orderInCategory="100"
      app:showAsAction="always" />

</menu>
```



Part 1.2 solution

```
@Override
public boolean onOptionsItemSelected(MenuItem item) {

    int id = item.getItemId();

    if (id == R.id.mySettings) {

        Intent i = new Intent(this, SettingsActivity.class);
        startActivity(i);

        return true;
    }

    return super.onOptionsItemSelected(item);
}
```



That's all we need to do
in the main activity 😊

Let's see part 2.

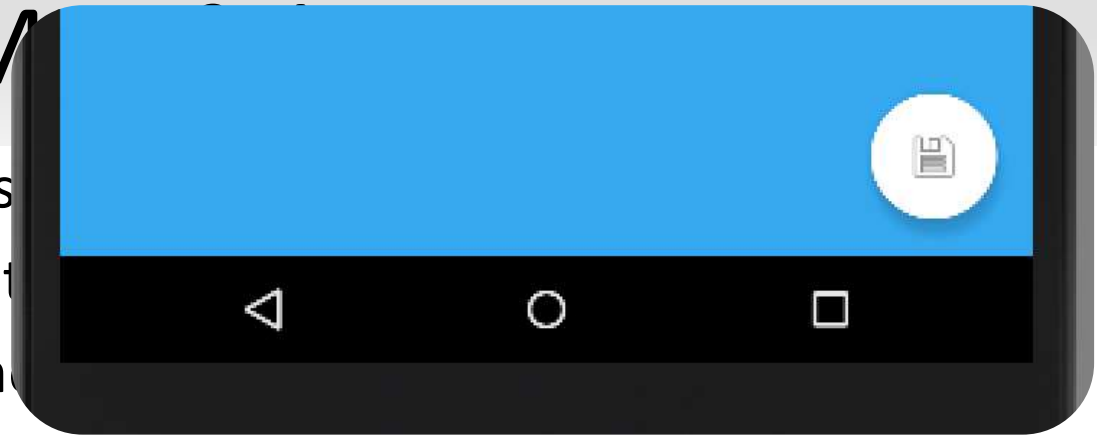


Part 2: the Settings Activity

- Part 2.1 design it in XML
- Part 2.2 Java:
 - 2.2.A) inside onCreate, try to read the sharedPreferences file and extract the values and set the editText and the switch accordingly. Be careful, it might be the first time you enter and nothing is saved yet.
 - 2.2.B) Program the floating button in a way that when clicked, the 2 values are extracted from the layout, and saved into the sharedPreferences file. Also make the snackbar appear to tell the user that save was successful. (Snackbar is like toast but more advanced)



2.1 XM



- Remember there are 2 files
- One is main and one is cont
- The floating button is in the

In the main, change this in the floating button

```
<android.support.design.widget.FloatingActionButt
on
    android:id="@+id/fab"
    android:layout_width="wrap_content"
android:layout_height="wrap_content"
    android:layout_gravity="bottom|end"
android:layout_margin="@dimen/fab_margin"
    android:src="@android:drawable/ic_menu_save"
    app:backgroundTint="#ffffff" />
```



2.1 XML of the content

<EditText

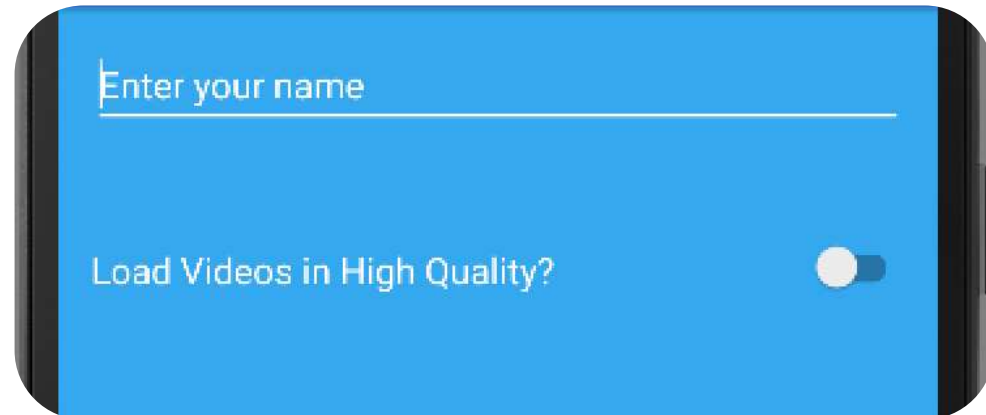
```
android:layout_width="wrap_content"  
android:layout_height="wrap_content"  
android:id="@+id/editText"  
android:layout_alignParentTop="true"  
android:hint="Enter your name"  
android:textColorHint="#ffffff"  
android:layout_alignParentStart="true"  
android:layout_alignParentEnd="true" />
```

<Switch

```
android:layout_width="wrap_content"  
android:layout_height="wrap_content"  
android:id="@+id/switch1"  
android:layout_below="@+id/editText"  
android:layout_alignParentEnd="true"  
android:layout_marginTop="53dp"  
android:checked="false" />
```

<TextView

```
android:layout_width="wrap_content"  
android:layout_height="wrap_content"  
android:text="Load Videos in High Quality?"  
android:id="@+id/textView"  
android:layout_alignBottom="@+id/switch1"  
android:layout_alignParentStart="true"  
android:textColor="#ffffff"  
android:textSize="18dp" />
```





JAVA part 2.2.A (displaying part)

- Remember: we need to read values from `sharedPref` and display them.
- Assume that we will name the `sharedpref` file “Settings”, the name of the user “user_name” and the switch boolean value as “high_quality”



2.2.A (solution)

```
protected void onCreate(Bundle savedInstanceState) {  
    // ... leave any code as is.  
    // add this to make the back arrow appear on the screen  
    getSupportActionBar().setDisplayHomeAsUpEnabled(true);  
  
    // grab the elements from the layout  
    e = findViewById(R.id.editText); // e is global editText  
    s = findViewById(R.id.switch1); // s is global switch  
  
    // grab shared pref  
    SharedPreferences display_sp =  
    getSharedPreferences("settings",Context.MODE_PRIVATE);  
  
    name = display_sp.getString("user_name",null);  
    // null is default value in case user_name is not created yet.  
    choice = display_sp.getBoolean("high_quality",false);  
    //false is default in case high_quality cannot be found.  
  
    if (name!=null) { e.setText(name); }  
  
    s.setChecked(choice);
```



2.2.B Programming the floating button

- Btw, we are still inside onCreate 😊

```
FloatingActionButton fab = findViewById(R.id.fab);  
fab.setOnClickListener(new View.OnClickListener() {  
    @Override  
    public void onClick(View view) {
```

```
        // open shared pref
```

```
        SharedPreferences sp = getSharedPreferences("settings", Context.MODE_PRIVATE);
```

```
        // read values from user
```

```
        name = e.getText().toString();
```

```
        choice = s.isChecked();
```





2.2.B Programming the floating button

- Continuing:

```
// create editor
```

```
SharedPreferences.Editor myEditor = sp.edit();
```

```
// save values
```

```
myEditor.putBoolean("high_quality",choice);
```

```
myEditor.putString("user_name",name);
```

```
// commit
```

```
myEditor.commit();
```

```
// show snackbar to tell user everything is saved.
```

```
Snackbar.make(view, "Save Successful", Snackbar.LENGTH_LONG).show();
```

```
}
```

```
});
```

```
} // on create ends here
```



Lab session SharedPreferences

